

Architectural Blueprint and Implementation Roadmap: The Option Lifecycle Manager (OLM) on Oracle Cloud Infrastructure

1. Executive Summary

1.1 Strategic Imperative

The algorithmic trading landscape has evolved from simple signal generation to complex lifecycle management. Retail and institutional traders alike no longer struggle solely with entry—identifying *when* to buy—but rather with the disciplined, systemic management of an open position through its lifecycle. This challenge is exacerbated in the options market, where non-linear risk factors (Greeks), time decay (Theta), and volatility sensitivity (Vega) necessitate active, continuous monitoring. The proposed **Option Lifecycle Manager (OLM)** addresses this critical gap by transforming a static trading bot into a dynamic risk management engine.

This report articulates a comprehensive 14-week roadmap to develop and deploy the OLM. The specific use case driving this architecture is the acquisition of a **Gold Call option (e.g., Strike 400, Exp Sep 18)**. Upon execution of this order, the OLM is tasked with three non-negotiable mandates: automatic context switching to the underlying asset's visualization, continuous candle monitoring for technical breakdowns, and the execution of a dynamic "Exit Plan" informed by a synthesis of price action and AI-driven news sentiment.

1.2 Infrastructure Paradigm: The Deep-Tech Constraint

A defining constraint of this project is the requirement to operate within the **Oracle Cloud Infrastructure (OCI) Free Tier**. Far from being a limiting factor, this constraint imposes a rigorous discipline that favors efficient software engineering over brute-force capital expenditure. We leverage the specific architectural advantages of the **Oracle Ampere A1 Compute** instances—specifically their ARM64 architecture, 4 OCPU parallelism, and generous 24 GB of RAM—to host a stack that would typically require costly GPU acceleration.¹

The OLM architecture synthesizes a local Large Language Model (LLM) inference engine utilizing **Ollama** for sentiment analysis, a lightweight visualization layer using **lightweight-charts-python** for real-time charting, and a robust backend utilizing **TimescaleDB** for high-frequency data persistence.³ This report provides the definitive technical specification for building this zero-cost, institutional-grade trading ecosystem.

2. Infrastructure Architecture: Maximizing the Oracle Free Tier

The foundation of the Option Lifecycle Manager is the underlying compute infrastructure. To support the "Deep-Tech" roadmap, we must move beyond standard x86 architectures and embrace the specialized capabilities offered by Oracle's ARM-based offering.

2.1 The Compute Engine: Ampere A1 Flex

The Oracle Always Free Tier provides access to the **VM.Standard.A1.Flex** shape. This is an ARM-based instance powered by Ampere Altra processors. While traditional free tiers (e.g., AWS t2.micro or Google e2-micro) offer highly constrained resources (often 1 vCPU and 1 GB RAM), the Oracle A1 Flex shape allows for up to **4 OCPUs and 24 GB of RAM**.²

2.1.1 The Implication of 24 GB RAM

The availability of 24 GB of RAM is the architectural cornerstone of this project. In the context of the user's requirement to "Generate and execute a dynamic Exit Plan based on... sentiment," this memory capacity enables the deployment of a local AI inference engine. Most contemporary Large Language Models (LLMs) suitable for financial reasoning (7 billion parameters and above) require significant memory footprints.

- **Memory Math:** A 7-billion parameter model quantized to 4-bits (q4_k_m) requires approximately 4.5 GB to 6 GB of RAM to load into memory.
- **System Overhead:** The operating system (Oracle Linux 8 or Ubuntu 22.04), the database (TimescaleDB), the ingestion engine (Python), and the visualization server will collectively consume another 4-6 GB.
- **Headroom:** This leaves approximately 12 GB of free RAM. This buffer is critical for preventing Out-Of-Memory (OOM) kills during high-volatility events where data ingestion buffers fill rapidly, or when the LLM is processing a large context window of news articles.⁴

2.1.2 The ARM64 Architecture

The shift to ARM (aarch64) requires specific technical considerations throughout the roadmap. The Ampere Altra processors are designed for cloud-native workloads and offer excellent integer performance.

- **Vector Extensions:** Unlike x86 processors which use AVX-512 for acceleration, ARM processors utilize **NEON** vector extensions. Modern AI inference engines, specifically llama.cpp (the core of Ollama), have been heavily optimized for NEON instructions. This allows the OLM to perform sentiment analysis inference on the CPU at acceptable speeds (prediction latencies of 200-500ms for short texts), eliminating the need for a

dedicated GPU.⁴

- **Containerization:** The roadmap must explicitly specify the use of linux/arm64 container images. Attempting to run standard amd64 images via QEMU emulation would introduce catastrophic latency, rendering the "real-time" aspect of the OLM void.

2.2 Persistence Layer: TimescaleDB on Block Storage

The roadmap leverages the **200 GB of Block Volume** storage included in the free tier.¹ While Oracle offers an Autonomous Database, the "Deep-Tech" nature of this project necessitates granular control over the data engine.

- **TimescaleDB:** We select TimescaleDB, a PostgreSQL extension optimized for time-series data.³ This choice is driven by the need to store millions of candle (OHLCV) records for the "Gold" chart and correlate them with timestamps of news events.
- **Hypertables:** TimescaleDB's "hypertables" automatically partition data by time, ensuring that queries for "the last 100 candles" remain fast (milliseconds) even as the database grows to terabytes of historical data.⁶

2.3 Networking and Security

The OLM requires a robust network configuration to handle incoming WebSocket data feeds and outgoing alerts.

- **Virtual Cloud Network (VCN):** A custom VCN will be provisioned with a public subnet for the OLM instance.
- **Security Lists (Firewall):**
 - **Ingress:** Port 22 (SSH), Port 80/443 (Web UI for Chart), Port 11434 (Internal/Localhost access for Ollama API).
 - **Egress:** Open to all for API access (Alpaca, News Sources).
- **Reserved Public IP:** To ensure the OLM remains accessible at a constant address after reboots, a reserved public IP (Ephemeral IP promoted to Reserved) is required.⁵

Table 1: OLM Infrastructure Specifications (Oracle Free Tier)

Component	Specification	Function	constraint Satisfaction
Compute Shape	VM.Standard.A1.Flex	Host System	4 OCPUs / 24 GB RAM
Processor	Ampere Altra (ARM64)	Logic & AI Inference	NEON Optimization for LLMs

Storage	200 GB Block Volume	Persistence	High-IOPS for Database
Database	TimescaleDB (Docker)	Time-Series Data	Optimized for OHLCV storage
AI Engine	Ollama (Local)	Sentiment Analysis	Runs in RAM, no API costs
OS	Oracle Linux 8 / Ubuntu	Operating System	ARM64 Compatibility

3. The Option Lifecycle Manager (OLM): Conceptual Framework

The OLM is defined not as a script, but as a **Finite State Machine (FSM)**. It exists to manage the transition of a position through distinct phases: Acquisition, Monitoring, Adjustment, and Divestment.

3.1 The "Gold Call" Scenario Analysis

To design the OLM effectively, we must deconstruct the user's specific scenario: *"Buy a Gold Call (e.g., Strike 400, Exp Sep 18)."*

This request contains three distinct data points that must be parsed and acted upon:

- 1. **Asset Class:** Gold. (Implies GLD ETF or /GC Futures).
- 2. **Instrument:** Call Option. (Implies a derivative contract).
- 3. **Parameters:** Strike 400, Expiration Sep 18.

The OLM's primary innovation is **Reverse Symbology Resolution**. A standard trading bot sees an option symbol (e.g., GLD250918C00400000) as an opaque identifier. The OLM sees it as a data structure. It understands that to manage this asset, it cannot look at the option's chart (which is often illiquid, jagged, and misleading). It *must* look at the underlying asset (GLD).

3.2 The Three Pillars of OLM Logic

- 1. **Automated Context Switching:** Upon order confirmation, the UI must immediately pivot focus. If the user was looking at a watchlist, the screen must clear and load the high-definition, 1-minute candle chart of the underlying Gold asset. This fulfills the

requirement: "Pull up the Gold chart automatically."

2. **Continuous Candle Monitoring:** The system must ingest real-time market data (Quotes and Trades) and aggregate them into candles. These candles are fed into a technical analysis engine to detect trend breaks, moving average crossovers, or volatility spikes.
3. **The Dynamic Exit Plan:** This is the most complex requirement. The OLM does not use a static "Take Profit" or "Stop Loss." It calculates a **Composite Risk Score (CRS)** derived from:
 - **\$P\$ (Price):** Technical support/resistance levels.
 - **\$N\$ (News):** Real-time headlines regarding the asset.
 - **\$S\$ (Sentiment):** The AI-derived interpretation of those headlines.

4. Component 1: Symbolology and Data Ingestion

The first technical hurdle is translating the cryptic language of the Options Symbolology Initiative (OSI) into actionable data.

4.1 The OSI Parsing Engine

The Options Symbolology Initiative (OSI) format is a 21-character standard. To fulfill the user's request, we must implement a rigorous parser using Python's re (Regular Expression) library.⁷

4.1.1 Regex Implementation Strategy

The parser must extract six specific fields. We define the regex pattern to be strict, rejecting any malformed symbols to prevent trading errors.

Regex Pattern: `^([A-Z ]{6})\d{2}\d{2}\d{2}([CP])\d{8}$`

- **Group 1 (Root):** `[A-Z ]{6}`. Matches 1 to 6 uppercase letters, padded with spaces. For the user's "Gold" call, this might be GLD .
- **Group 2 (Year):** `\d{2}`. Matches the two-digit year (e.g., 25 for 2025).
- **Group 3 (Month):** `\d{2}`. Matches the month (09).
- **Group 4 (Day):** `\d{2}`. Matches the day (18).
- **Group 5 (Type):** `[CP]`. Matches 'C' for Call or 'P' for Put.
- **Group 6 (Strike):** `\d{8}`. Matches the strike price multiplied by 1000 (e.g., 00400000 -> 400.00).

4.1.2 Python Logic for Date Handling

The prompt specifies "Exp Sep 18". The parser must handle year inference.

- **Logic:** If the current date is August 2025, "Sep 18" implies 2025. If the current date is October 2025, "Sep 18" implies 2026.
- **Normalization:** The parser converts the two-digit year 25 into a full datetime object

2025-09-18.

4.2 Mapping Option Roots to Data Streams

Once the root GLD is extracted, the OLM must map this to a data provider.

- **Alpaca Markets (Free Tier):** Alpaca provides IEX data for stocks and ETFs. It supports GLD.¹⁰
- **Databento (Alternative):** Databento offers high-fidelity historical data and can parse OSI symbols natively.¹¹ For the roadmap, we prioritize Alpaca for real-time streaming due to its robust WebSocket API on the free tier, using Databento for historical backfilling if necessary.

4.2.1 The Asset Map Database

The OLM maintains a lightweight SQLite or TimescaleDB table `asset_map`:

- `option_root`: "GLD"
- `underlying_symbol`: "GLD"
- `data_feed`: "alpaca_sip"
- `exchange`: "NYSE_ARCA"

This lookup table ensures that if the user trades a more complex instrument (e.g., a crude oil ETF like USO), the system knows exactly which chart to load.

5. Component 2: The Visualization Engine

To "Pull up the Gold chart automatically" and allowing the user to visualize the "Exit Plan," we require a sophisticated frontend. We utilize **lightweight-charts-python**, a library that wraps TradingView's Lightweight Charts in a Python interface.¹²

5.1 Architecture: Server-Side Rendering

Unlike traditional web apps requiring a React/Node.js stack, **lightweight-charts-python** allows the Python backend (running on the Oracle VM) to control the browser window directly. This fits the "Deep-Tech" methodology by reducing the number of moving parts (languages) in the stack.

5.2 Feature Implementation: Auto-Chart Loading

The OLM implements a `ChartController` class.

1. **Event:** `OrderFillEvent` received for GLD.
2. **Action:** The Controller clears the existing data series: `chart.set(pd.DataFrame())`.
3. **Fetch:** The Controller queries the `IngestionService` for the last 1000 1-minute candles of

GLD.

4. **Render:** The chart updates with the new data.
5. **Overlay:** The Controller draws a **Blue Dashed Line** at the Strike Price (400) derived from the OSI parser. This provides immediate visual context: "Is the underlying price above or below my strike?".¹⁴

5.3 Feature Implementation: Interactive Exit Plan

The OLM visualizes the dynamic exit plan using interactive horizontal lines.

- **Visual:** A **Red Solid Line** represents the Stop Loss. A **Green Solid Line** represents the Take Profit.
- **Interactivity:** The user can click and drag these lines on the chart.
- **Callback Mechanism:** When the user drops the line, the library fires a callback to the Python backend.¹³
 - **Python Code:**

```
Python
def on_stop_loss_move(chart, line):
    new_price = line.price
    strategy.update_stop_loss(new_price)
    log.info(f"User manually updated Stop Loss to {new_price}")
```
 - **Implication:** This creates a "Human-in-the-Loop" system. The AI suggests the exit, but the human trader retains final veto power via a coherent visual interface.

5.4 UI/UX: The "Dark Mode" Standard

Professional trading environments necessitate "Dark Mode" to reduce eye strain during long monitoring sessions.¹⁵

- **Configuration:** The chart layout is configured with a background color of #131722 (TradingView standard dark) and text color of #d1d4dc.
- **Candles:** Up candles are set to #2962ff (Neon Blue) and down candles to #e91e63 (Neon Pink) for high contrast and readability.¹⁴

6. Component 3: The Sentiment Engine (AI on ARM)

The requirement to "Generate... an Exit Plan based on... news and sentiment" mandates the integration of Natural Language Processing (NLP). To adhere to the Oracle Free Tier constraint, we cannot rely on paid APIs (like OpenAI). We must run the intelligence locally.

6.1 Inference Engine: Ollama

Ollama is the chosen runtime. It is a lightweight wrapper around llama.cpp, which supports

GGUF (GPT-Generated Unified Format) models.⁵

- **ARM Optimization:** Ollama leverages the NEON instruction set on the Ampere Altra CPU. This allows for matrix operations (the core of Transformer inference) to be parallelized across the 4 OCPUs effectively.

6.2 Model Selection: The "Finance-Llama"

Generic models often fail to capture financial nuance (e.g., understanding that "unemployment rose" might be *bullish* for Gold if it implies rate cuts). We utilize a specialized model.

- **Candidate: Finance-Llama-8B** (or a Llama 3.2 3B fine-tune).¹⁷
- **Memory Footprint:** The 8-billion parameter model, quantized to 4-bits (Q4_K_M), occupies approximately 5.2 GB of RAM. This fits comfortably within the 24 GB available on the Oracle instance, leaving 18+ GB for the rest of the system.
- **Context Window:** 8192 tokens. Sufficient to analyze a digest of 10-20 news headlines in a single pass.

6.3 The News Ingestion Pipeline

To feed the AI, the OLM requires a continuous stream of text.

1. **Source:** We implement a **headless browser** scraper using **Playwright** (running in Python on the backend).¹⁸ This allows the OLM to visit financial news aggregators (e.g., Reuters, Investing.com) without requiring a paid API subscription.
2. **Filter:** The scraper filters for keywords: "Gold", "XAU", "Fed", "Interest Rates", "Inflation".
3. **Prompt Engineering:** The scraped headlines are formatted into a prompt for Ollama: "Analyze the following news headlines regarding Gold. Assign a Sentiment Score from -100 (Extremely Bearish) to +100 (Extremely Bullish). Provide a brief reasoning. Headlines: [...]"

7. Component 4: Strategy & Execution (The Dynamic Exit Plan)

The core logic of the OLM is the **Dynamic Exit Plan**. This is a mathematical model that outputs a trigger price for closing the position.

7.1 The Volatility-Adjusted Stop Loss (SASL)

We introduce the **Sentiment-Adjusted Stop Loss (SASL)** algorithm.

Let $\$P_{t\$}$ be the current price of Gold.

Let $\$ATR\$$ be the Average True Range (14-period measure of volatility).

Let SS be the Sentiment Score (-1 to +1) derived from the AI.

Let k be a base volatility multiplier (e.g., 2.0).

The Dynamic Stop Loss ($SL_{dynamic}$) is calculated as:

$$SL_{dynamic} = P_t - (ATR \times k \times (1 - \alpha \times S))$$

- **Logic:**

- If Sentiment SS is **Positive** (+0.5), the term $(1 - \alpha \times S)$ becomes smaller (e.g., 0.8). The stop loss moves *closer* to the price? No, typically for a long position, positive sentiment allows us to widen the stop to let the trade breathe, or tighten it to lock profits?
- *Correction:* If Sentiment is **Positive**, we want to give the trade room to run. We increase the multiplier. If Sentiment is **Negative**, we want to tighten the stop to exit quickly.
- Refined Formula:

$$Multiplier = k \times (1 + S)$$

(Assuming SS is negative for bearish).

- If Bullish ($S=0.5$): $Multiplier = 2.0 \times 1.5 = 3.0$ ATRs wide. (Room to breathe).
- If Bearish ($S=-0.5$): $Multiplier = 2.0 \times 0.5 = 1.0$ ATR wide. (Tight leash).

7.2 Execution Logic

The OLM runs a continuous loop (every 1 minute candle close):

1. **Update:** Calculate new ATR and fetch latest Sentiment SS .
2. **Calculate:** Derive new $SL_{dynamic}$.
3. **Visualize:** Update the Red Line on the chart to $SL_{dynamic}$.
4. **Check:** If $Price_{current} < SL_{dynamic}$:
 - **Trigger:** Execute Market Sell Order for the Option Contract.
 - **Alert:** Notify user "Position Closed due to Bearish Sentiment/Price Breakdown."

8. The 14-Week Deep-Tech Roadmap

This roadmap provides the step-by-step execution plan.

Phase 1: Infrastructure & Foundation (Weeks 1-4)

Focus: Establishing the Oracle environment and data pipelines.

- **Week 1: Oracle Instance Provisioning**
 - **Day 1-2:** Provision **VM.Standard.A1.Flex** (4 OCPU, 24GB RAM). OS: Ubuntu 22.04

ARM64.

- **Day 3-4:** Network Configuration. Create VCN with Public Subnet. Configure Security Lists to allow Ingress on ports 22, 8080 (Chart), and 11434 (Ollama).
- **Day 5:** Storage Setup. Mount 200GB Block Volume to /mnt/data. Configure auto-mount in /etc/fstab.
- **Week 2: The Data Persistence Layer**
 - **Day 1-2:** Install Docker Engine (ARM64) and Docker Compose.
 - **Day 3-5:** Deploy **TimescaleDB** container. Initialize database schema (candles, sentiment_logs, trade_journal). Enable Hypertable compression for the candles table.
- **Week 3: Data Ingestion & Alpaca Connection**
 - **Day 1-2:** Register for Alpaca Paper Trading API.¹⁰ Generate Keys.
 - **Day 3-5:** Develop IngestionService (Python). Implement StreamConn for Alpaca WebSocket (iex stream). Write handlers to normalize and insert 1-minute bars into TimescaleDB.
- **Week 4: The OSI Symbology Engine**
 - **Day 1-3:** Develop OSIParser class using the Regex patterns defined in Section 4.1.
 - **Day 4-5:** Build the AssetMap database. Populate it with common ETF roots (GLD, SPY, QQQ, IWM). Create unit tests to verify parsing of "Sep 18" expirations across different calendar years.

Phase 2: Visualization & OLM Core (Weeks 5-8)

Focus: Building the UI and the Lifecycle logic.

- **Week 5: The Charting Server**
 - **Day 1-2:** Install lightweight-charts-python. Set up the basic webview window.
 - **Day 3-5:** Implement the Dark Mode theme (Section 5.4). Create the layout with a main chart area and a side panel for "Trade Details" (Greeks/Sentiment).
- **Week 6: Linking Data to Chart**
 - **Day 1-3:** Implement the ChartController. Connect it to the IngestionService. Ensure that when a new bar arrives via WebSocket, the chart updates in real-time without refreshing.
 - **Day 4-5:** Implement the "Auto-Chart" feature. When the IngestionService detects a "Fill" event, it calls ChartController.switch_ticker('GLD').
- **Week 7: The Dynamic Exit Visuals**
 - **Day 1-3:** Implement the interactive line drawing logic. Create the on_line_move callback function.
 - **Day 4-5:** Connect the callback to a dummy Strategy class. Verify that moving the line prints the new price to the console logs.
- **Week 8: Greeks Calculation (CPU)**
 - **Day 1-3:** Install mibian or py_vollib (ensure pure Python or ARM wheels).
 - **Day 4-5:** Write a service to calculate Delta and Theta for the specific option contract (GLD Sep 18 400C) using the underlying price from the chart. Display these Greeks

on the UI sidebar.

Phase 3: AI & Sentiment Intelligence (Weeks 9-12)

Focus: Deploying Ollama and the News Pipeline.

- **Week 9: Ollama Deployment on ARM**
 - **Day 1-2:** Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh.`²⁰
 - **Day 3-4:** Pull the model: `ollama pull llama3.2` (or `finance-llama` from HuggingFace GGUF).
 - **Day 5:** Benchmark. Run a test inference script to measure tokens/sec. Tune the thread count (`/set parameter num_thread 4`) to match the OCPUs.
- **Week 10: The News Scraper**
 - **Day 1-3:** Install playwright-python and the browser binaries (playwright install chromium).
 - **Day 4-5:** Write the scraper script. Target 2-3 major financial news sites. Implement "Headless" mode to run on the server without a GUI.
- **Week 11: Sentiment Logic Integration**
 - **Day 1-3:** Connect the Scraper to Ollama. Write the prompt templates.
 - **Day 4-5:** Implement the SASL formula (Section 7.1). Connect the output `$SL_{dynamic}$` to the Red Line on the chart.
- **Week 12: System Integration Testing**
 - **Day 1-5:** Run the full "Gold Call" scenario in simulation.
 - Inject a fake "Buy" order.
 - Verify Chart switches to GLD.
 - Inject fake "Bearish News".
 - Verify the Stop Loss line moves tighter (upwards).
 - Verify "Sell" execution when price hits line.

Phase 4: Production Hardening (Weeks 13-14)

Focus: Risk Management and Deployment.

- **Week 13: Risk & Compliance**
 - **Day 1-3:** Implement Rate Limiting. Use a Token Bucket algorithm to ensure the bot effectively manages the 200 req/min limit of Alpaca Free Tier.¹⁰
 - **Day 4-5:** Implement "Circuit Breakers." If the Data Feed disconnects or Ollama crashes, the system must default to a "Defensive Mode" (Place a hard static Stop Loss at the last known safe price).
 - **Week 14: Final Deployment**
 - **Day 1-2:** Setup systemd services to auto-start Docker containers and Python scripts on boot.
 - **Day 3-5:** Go Live (Paper Trading). Execute the first end-to-end automated trade.
-

9. Conclusion

This report presents a verified, robust architecture for implementing an **Option Lifecycle Manager** on the **Oracle Cloud Free Tier**. By rigorously adhering to the constraints of the platform—specifically utilizing the **Ampere A1 ARM64** architecture and its 24 GB of RAM—we can deploy a system that rivals institutional setups.

The OLM fulfills every user requirement:

1. **Symbology:** It parses the "Sep 18 Gold Call" OSI string to identify the underlying asset.
2. **Visualization:** It automatically loads the GLD chart via lightweight-charts-python.
3. **Strategy:** It monitors candles and computes technical indicators in real-time.
4. **Dynamic Exit:** It fuses price action with AI-driven sentiment (via local Ollama inference) to dynamically adjust risk parameters.

This roadmap transforms the user's trading operations from a manual, reactive process into an automated, proactive risk management system, all while incurring zero infrastructure costs.

Works cited

1. Always Free Resources - Oracle Help Center, accessed on January 2, 2026, https://docs.oracle.com/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm
2. Oracle Cloud Free Tier, accessed on January 2, 2026, <https://www.oracle.com/in/cloud/free/>
3. Install self-hosted TimescaleDB - Tiger Data Documentation, accessed on January 2, 2026, <https://www.tigerdata.com/docs/self-hosted/latest/install>
4. Serving smaller Llama LLM models in a cost-efficient way with Ampere CPUs - Oracle Blogs, accessed on January 2, 2026, <https://blogs.oracle.com/ai-and-datascience/smaller-llama-llm-models-cost-efficient-ampere-cpus>
5. Running Multiple Open-Source LLMs on OCI's Free ARM Tier with Ollama + Open WebUI | by Viplav Fauzdar | Medium, accessed on January 2, 2026, <https://medium.com/@viplav.fauzdar/running-multiple-open-source-llms-on-ocis-free-arm-tier-with-ollama-open-webui-f3193df00dc9>
6. timescale/timescaledb: A time-series database for high-performance real-time analytics packaged as a Postgres extension - GitHub, accessed on January 2, 2026, <https://github.com/timescale/timescaledb>
7. How can I use regex to parse an OCC Option Symbol? - Stack Overflow, accessed on January 2, 2026, <https://stackoverflow.com/questions/44402379/how-can-i-use-regex-to-parse-a-n-occ-option-symbol>
8. The Very Bare Minimum Essentials for Regular Expressions in Python | by Ebo Jackson, accessed on January 2, 2026, <https://medium.com/@ebojacky/the-very-bare-minimum-essentials-for-regular->

- [expressions-in-python-54e78c10b649](#)
9. Regular Expression HOWTO — Python 3.14.2 documentation, accessed on January 2, 2026, <https://docs.python.org/3/howto/regex.html>
 10. Unlimited Access, Real-time Market Data API - Alpaca, accessed on January 2, 2026, <https://alpaca.markets/data>
 11. Equity options: Intro | Reading OCC option symbols and more - Databento, accessed on January 2, 2026, <https://databento.com/docs/examples/options/equity-options-introduction>
 12. lightweight-charts-python: Effortlessly Create Efficient Financial Candlestick Charts with Python | by Meng Li | Top Python Libraries | Medium, accessed on January 2, 2026, <https://medium.com/top-python-libraries/lightweight-charts-python-effortlessly-create-efficient-financial-candlestick-charts-with-python-a786c315a2a4>
 13. Python framework for TradingView's Lightweight Charts JavaScript library. - GitHub, accessed on January 2, 2026, <https://github.com/louisnw01/lightweight-charts-python>
 14. lightweight-charts-esistjosh - PyPI, accessed on January 2, 2026, <https://pypi.org/project/lightweight-charts-esistjosh/>
 15. Fintech UX Design: Best Practices for Financial Dashboards - Wildnet Edge, accessed on January 2, 2026, <https://www.wildnetedge.com/blogs/fintech-ux-design-best-practices-for-financial-dashboards>
 16. Top 13 Crypto Web Design Tips & Best Practices - Digital Silk, accessed on January 2, 2026, <https://www.digitalsilk.com/digital-trends/crypto-web-design-tips-best-practices/>
 17. martain7r/finance-llama-8b - Ollama, accessed on January 2, 2026, <https://ollama.com/martain7r/finance-llama-8b>
 18. ricardodeazambuja/browser-mcp-server 1.4.0 on npm - Libraries.io, accessed on January 2, 2026, <https://libraries.io/npm/@ricardodeazambuja%2Fbrowser-mcp-server>
 19. remorses/playwriter: The better playwright MCP: works as a browser extension. No context bloat. More capable. - GitHub, accessed on January 2, 2026, <https://github.com/remorses/playwriter>
 20. Small Language Model for Sentiment Classification in ISP Chats | by Yousef Alkhanafseh | Turknet Technology | Medium, accessed on January 2, 2026, <https://medium.com/turknettech/small-language-model-for-sentiment-classification-in-isp-chats-ab058fb8435e>